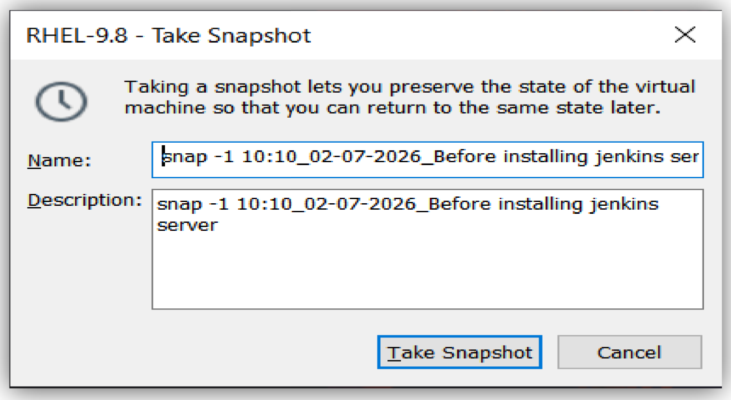
1. **Check the Change Request**
    - Check whether the change is assigned to you by the TL or manager.
    - Check the change status:
        - **Approved**
        - Pending
    - If the change is approved, then start the work.

2. **Access the Server**
    - Take the server IP from the TL.
    - Login using:
        - **SSH**
        - RDP
        - VNC
        - Remote Console

3. **Take Backup**
    - Always take backup before making changes.
    - Backup types:
        - Physical Server → NetBackup (Full OS Backup)
        - **Virtual Machine → Snapshot Backup**



        - AWS Instance → Snapshot Backup

4. **Configure Network and Hostname**
    - **Network Adapter: -** NAT
    - **Configure Static IP:-**
      [root@dhcpsrv1 ~]# nmcli con mod ens33 ipv4.method auto
      [root@dhcpsrv1 ~]# nmcli con up ens33

```
[root@localhost ~]# ifconfig ens33
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.5.146  netmask 255.255.255.0  broadcast 192.168.5.255
        inet6 fe80::20c:29ff:fed0:9028  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:d0:90:28  txqueuelen 1000  (Ethernet)
        RX packets 348  bytes 49253 (48.0 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 365  bytes 54234 (52.9 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

    - **Set the hostname: -**

[root@localhost ~]# hostnamectl set-hostname jenkins.radical.com

- **Check the hostname :-**
  [root@localhost ~]# hostname

- **Configure local DNS in /etc/hosts.**
  [root@jenkins ~]# vi /etc/hosts

```
[root@jenkins ~]# cat /etc/hosts
127.0.0.1   localhost localhost.localdomain localhost4 localhost4.localdomain4
::1         localhost localhost.localdomain localhost6 localhost6.localdomain6
192.168.5.146   jenkins.radical.com     jenkins
```

- **Test IP, hostname, and internet connectivity.**
  [root@jenkins ~] # ping 192.168.5.146
  [root@jenkins ~] # ping www.google.com
  [root@jenkins ~] # ping www.redhat.com

```
[root@jenkins ~]# ping 192.168.5.146
PING 192.168.5.146 (192.168.5.146) 56(84) bytes of data.
64 bytes from 192.168.5.146: icmp_seq=1 ttl=64 time=0.069 ms
64 bytes from 192.168.5.146: icmp_seq=2 ttl=64 time=0.115 ms
64 bytes from 192.168.5.146: icmp_seq=3 ttl=64 time=0.120 ms
^C
--- 192.168.5.146 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2081ms
rtt min/avg/max/mdev = 0.069/0.101/0.120/0.022 ms
[root@jenkins ~]#
[root@jenkins ~]#
[root@jenkins ~]# ping www.google.com
PING www.google.com (142.251.150.119) 56(84) bytes of data.
64 bytes from 142.251.150.119 (142.251.150.119): icmp_seq=1 ttl=128 time=10.4 ms
64 bytes from 142.251.150.119 (142.251.150.119): icmp_seq=2 ttl=128 time=21.1 ms
^C
--- www.google.com ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1010ms
rtt min/avg/max/mdev = 10.436/15.746/21.056/5.310 ms
[root@jenkins ~]#
[root@jenkins ~]#
[root@jenkins ~]# ping www.redhat.com
PING e3396.dscx.akamaiedge.net (23.38.58.41) 56(84) bytes of data.
64 bytes from a23-38-58-41.deploy.static.akamaitechnologies.com (23.38.58.41): icmp_seq=1 ttl=128 time=30.4 ms
64 bytes from a23-38-58-41.deploy.static.akamaitechnologies.com (23.38.58.41): icmp_seq=2 ttl=128 time=43.4 ms
^C
--- e3396.dscx.akamaiedge.net ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1009ms
rtt min/avg/max/mdev = 30.387/36.871/43.355/6.484 ms
```

5. **Configure Repository**

- **Local Repository**
  [root@jenkins ~]# mount /dev/sr0 /mnt

  [root@jenkins ~] # vi /etc/yum.repos.d/local.repo

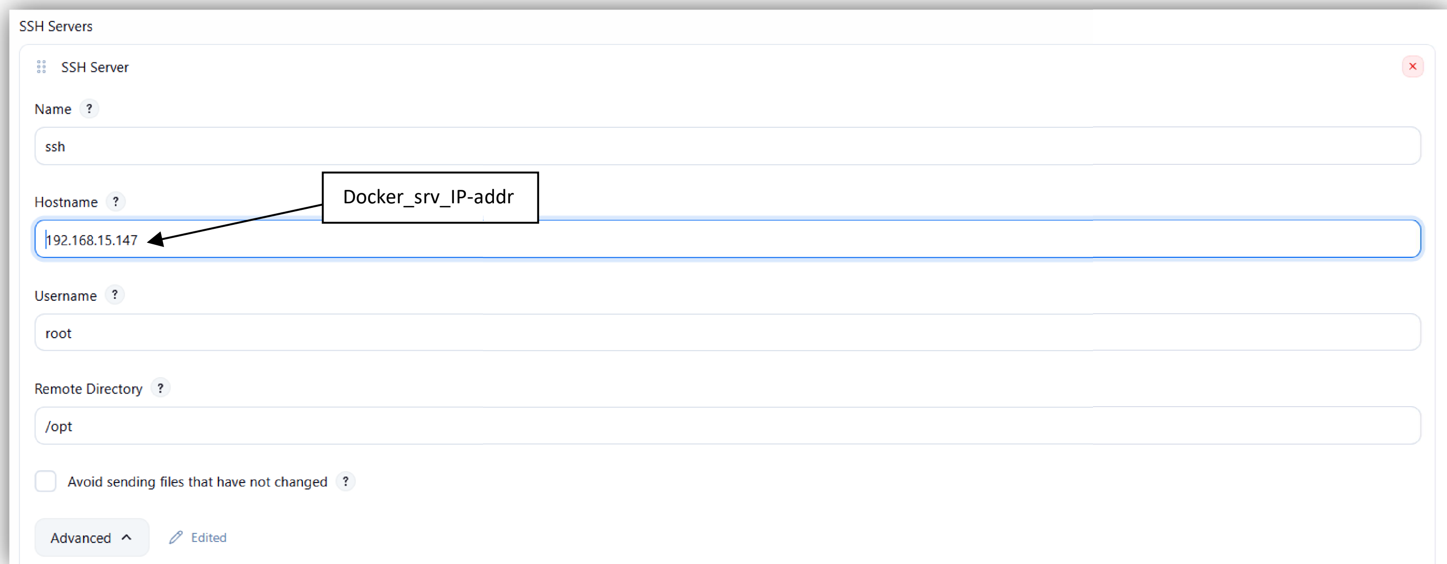
```
[AppStreamrepo]
name=this is AppStream file base repo
baseurl=file:///mnt/AppStream
enabled=1
gpgcheck=0


[BaseOS]
name=this is BaseOS file base repo
baseurl=file:///mnt/BaseOS
enabled=1
gpgcheck=0
```

- RHN Repository
- EPEL Repository
- Third-party Repository
- Satellite Server Repository

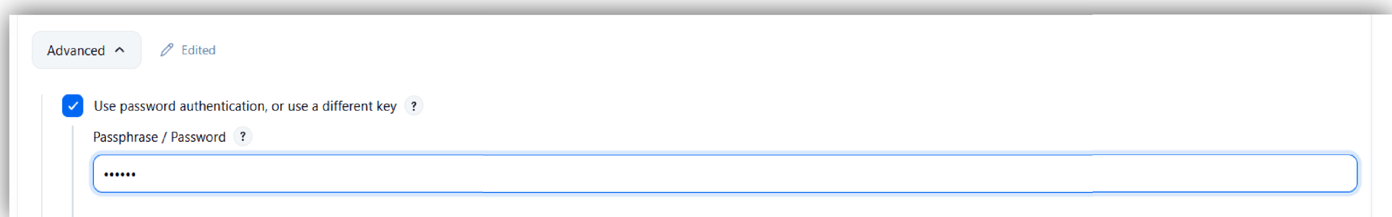
6. Verify and Install Required Packages :-

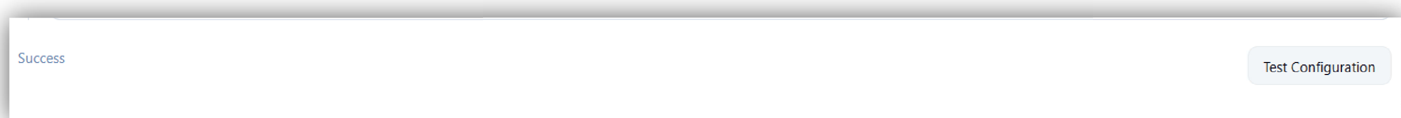
- Jenkins server already configured (installed :- java-21,maven,git, Jenkins-core)
- go to browser on jenkins server (Login)

  * go to setting
    * click on system
    * go down
  * click on add ssh



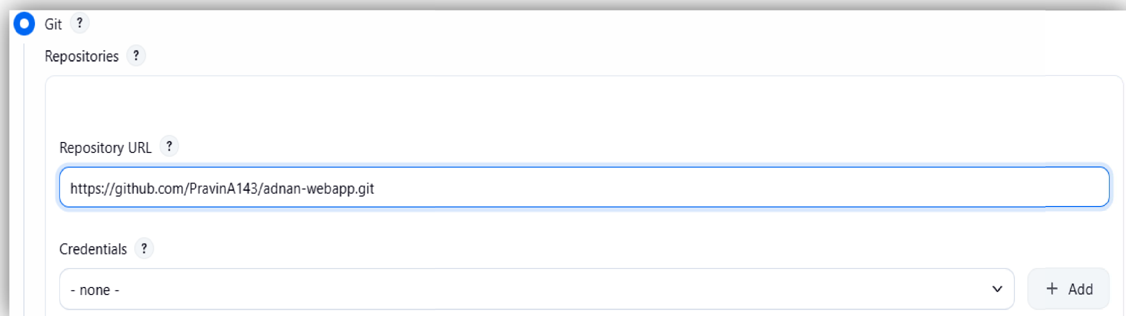
- click on advance



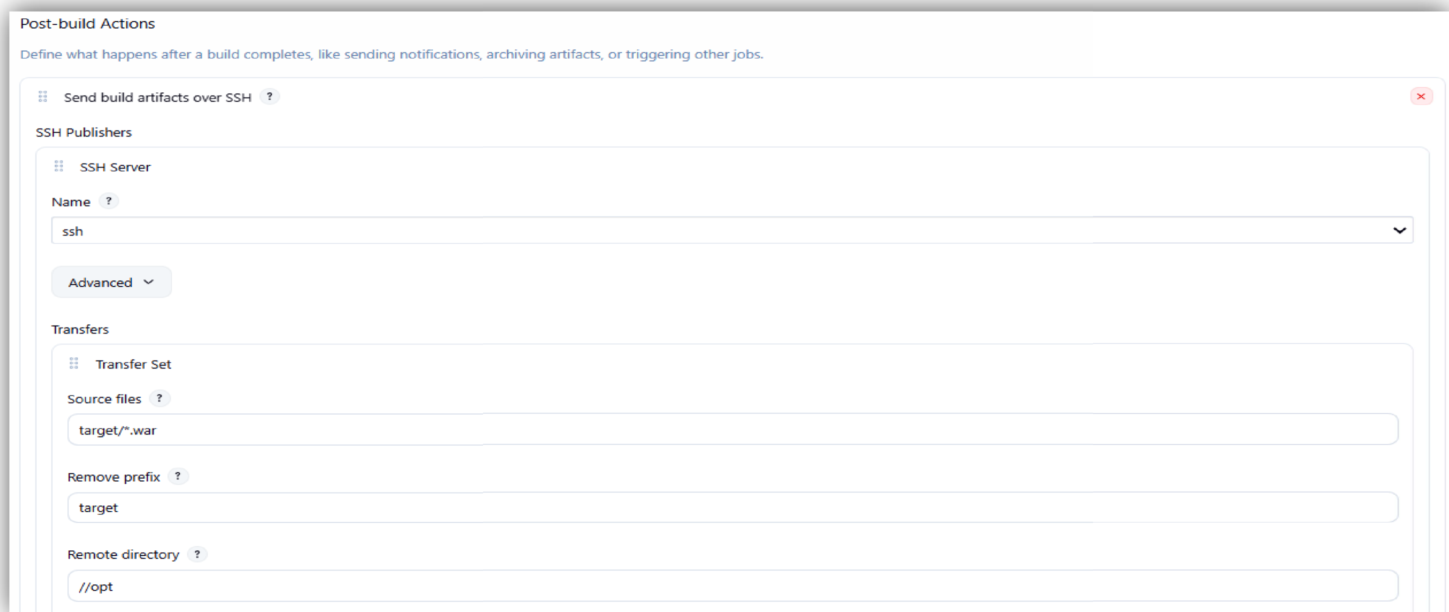


- **Apply and save**

- **Now, create item (job)**



Branch :- main
Poll SCM :- * * * * *
Goals and options :- clean install



Apply and save

- Click on build now ------ > output --- > Success

Now, go to another machine and configure docker_tomcat-container

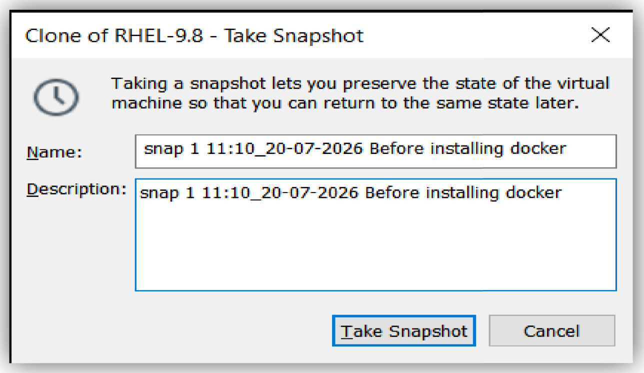
1. Access the Server

- Take the server IP from the TL.
- Login using:
  - **SSH**
  - RDP
  - VNC
  - Remote Console

2. Take Backup

- Always take backup before making changes.
- Backup types:
  - Physical Server → NetBackup (Full OS Backup)

- **Virtual Machine → Snapshot Backup**



- AWS Instance → Snapshot Backup

3. Configure Network and Hostname

- **Network Adopter: -** NAT
- **Configure Static IP:-**

[root@dhcpsrv1 ~]# nmcli con mod ens33 ipv4.method auto
[root@dhcpsrv1 ~]# nmcli con up ens33

```
[root@localhost ~]# ifconfig ens33
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.5.147  netmask 255.255.255.0  broadcast 192.168.5.255
        inet6 fe80::20c:29ff:fe19:dc43  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:19:dc:43  txqueuelen 1000  (Ethernet)
        RX packets 992  bytes 104088 (101.6 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 643  bytes 82902  (80.9 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

- **Set the hostname: -**

[root@localhost ~] # hostnamectl set-hostname docker.container.tomcat.radical.com

- **Check the hostname :-**

[root@docker ~] # hostname

- **Configure local DNS in /etc/hosts.**

[root@docker ~] # vi /etc/hosts

```
[root@docker ~]# cat /etc/hosts
127.0.0.1   localhost localhost.localdomain localhost4 localhost4.localdomain4
::1         localhost localhost.localdomain localhost6 localhost6.localdomain6
192.168.5.147   docker.radical.com      docker
```

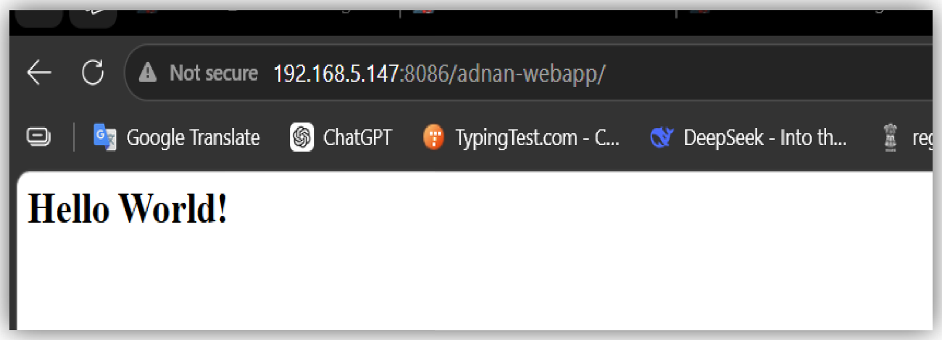
- **Test IP, hostname, and internet connectivity.**

[root@docker ~] # ping 192.168.5.147
[root@docker ~] # ping www.google.com
[root@docker ~] # ping www.redhat.com

- **Verify/Search/Install packages :-**
  First install java & tomcat
  # yum search java
  # yum install -y java-21-openjdk.x86_64

java --version
wget https://dlcdn.apache.org/tomcat/tomcat-10/v10.1.57/bin/apache-tomcat-10.1.57.tar.gz
tar -xvf apache-tomcat-10.1.57.tar.gz
mv apache-tomcat-10.1.57 tomcat
mv apache-tomcat-10.1.57.tar.gz /tmp/
cd /opt/tomcat/bin
./startup.sh
cp -a /opt/adnan-webapp.war /opt/tomcat/webapps/

- Now go to browser and put ip_address+port_number+web-app_name(192.168.5.147:8080/adnan-webapp)



- Now, install docker follow the steps

```
# dnf remove –y docker \
          docker-client \
          docker-client-latest \
          docker-common \
          docker-latest \
          docker-latest-logrotate \
          docker-logrotate \
          docker-engine \
          podman \
          runc
# dnf config-manager --add-repo https://download.docker.com/linux/rhel/docker-ce.repo
# dnf install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-  plugin
# systemctl enable --now docker.service
# docker pull tomcat
# docker images
# cd /opt
# vi dockerfile
```

```
[root@docker ~]# cat dockerfile
FROM tomcat:latest
MAINTAINER adnan
RUN  cp -R /usr/local/tomcat/webapps.dist/*   /usr/local/tomcat/webapps
COPY   ./*.war /usr/local/tomcat/webapps
```

docker build -t tomcat:fixed .
docker images
docker run -d --name web-c1 -p 8086:8080 tomcat:fixed
docker ps
go to browser search -- >http://192.168.5.147:8086 and http://192.168.5.:8086/adnan-webapp)

go to jenkins server install and configure the ansible-core

- **First create ssh_trust**
  # ssh-keygen -t rsa
  # ssh-copy-id 192.168.5.147 ----- > docker_Server ip
  # ssh 192.168.5.147 ----- > Verify

Now, install ansible package

yum search ansible
yum install -y ansible-core.x86_64
vi /etc/ansible/hosts

```
############ below setting done by adnan ###########

[docker_srv]
192.168.5.147
```

ansible all -m ping

```
[root@jenkins .ssh]# ansible all -m ping
192.168.5.147 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
[root@jenkins .ssh]#
```

cd /etc/ansible/
pwd --- > /etc/ansible/
vi mydockerplaybook.yml

```
---
- name: Build image and deploy image in container
  hosts: all
  become: yes
```

```
tasks:
  # stop docker container
  - name: stop existing docker container
    command: docker stop web-c1

  # clean and delete stopped docker container
  - name: delete docker container
    command: docker rm web-c1

  # clean and delete stopped docker image
  - name: delete docker image
    command: docker rmi tomcat:fixed

  # build docker image
  - name: build docker image
    command: docker build -t tomcat:fixed /opt/ -f /opt/dockerfile

  # start Docker container with new image
  - name: start new docker container
command: docker run -d --name web-c1 -p 8086:8080 tomcat:fixed
```

ansible-playbook mydockerplaybook.yml --syntax-check

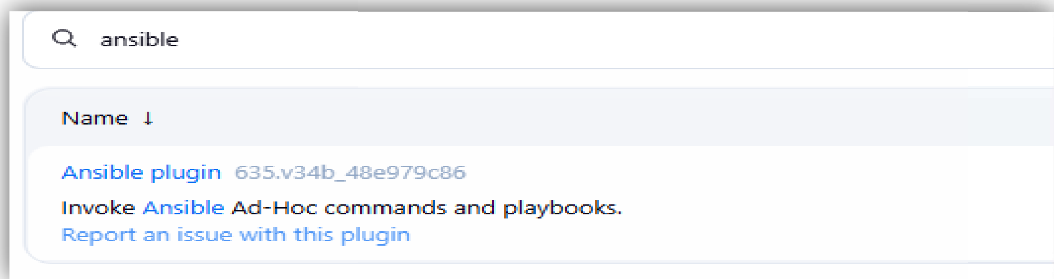
```
[root@jenkins ansible]# ansible-playbook mydockerplaybook.yml --syntax-check

playbook: mydockerplaybook.yml
```

ansible-playbook mydockerplaybook.yml –C ------- > dry run
ansible-playbook mydockerplaybook.yml

```
[root@jenkins ansible]# ansible-playbook mydockerplaybook.yml

PLAY [Build image and deploy image in container] ****************************************************************

TASK [Gathering Facts] ******************************************************************************************
ok: [192.168.5.147]

TASK [stop existing docker container] ***************************************************************************
changed: [192.168.5.147]

TASK [delete docker container] **********************************************************************************
changed: [192.168.5.147]

TASK [delete docker image] **************************************************************************************
changed: [192.168.5.147]

TASK [build docker image] ***************************************************************************************
changed: [192.168.5.147]

TASK [start new docker container] *******************************************************************************
changed: [192.168.5.147]

PLAY RECAP ******************************************************************************************************
192.168.5.147              : ok=6    changed=5    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

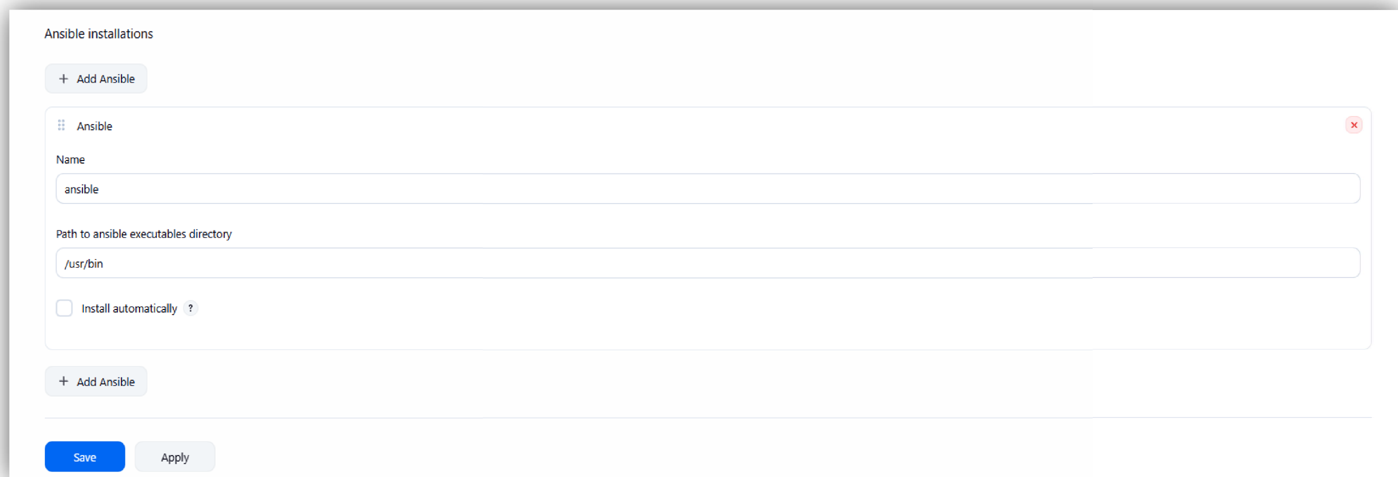
- **go to browser jenkins server**

go to setting ----- > plugins ----- > install ansible plugin

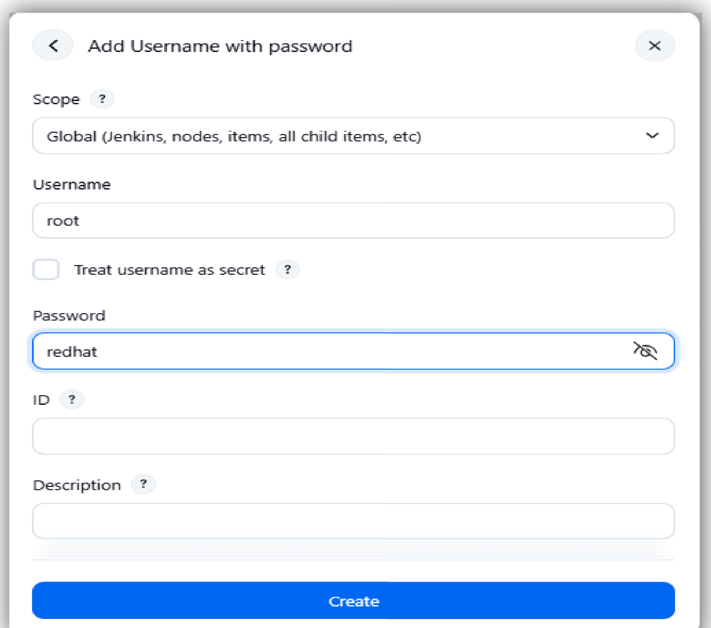


restart the jenkins (systemctl restart jenkins)

go to setting --- > tools
* go down

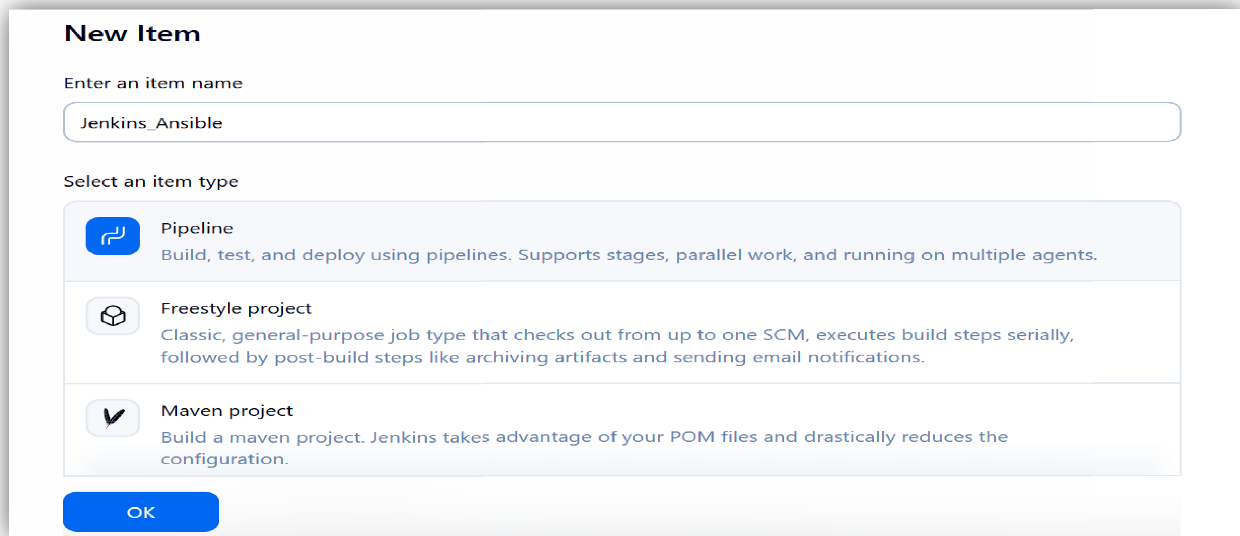


go to setting ---- > credentials
* add credentials
* Select a type of credential :- username with password

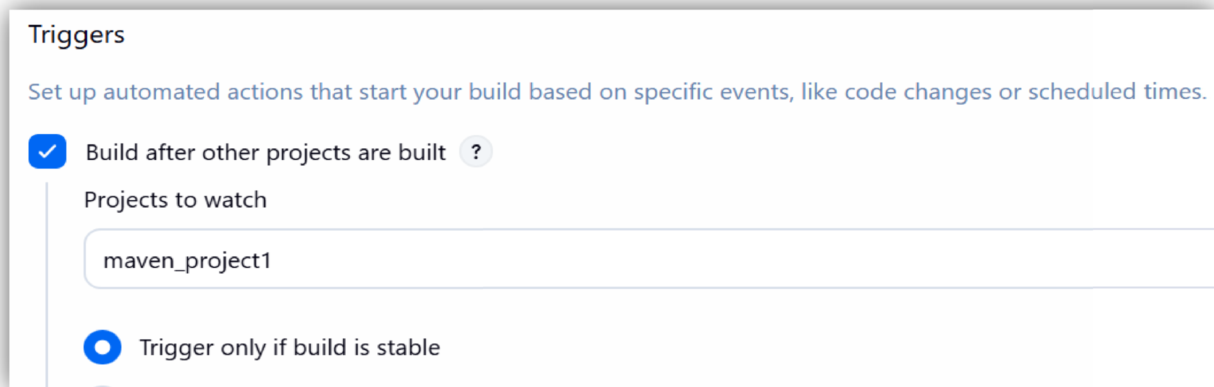


** setting ---- > system ------ > ssh_server ----- > already created

** create new item (job)

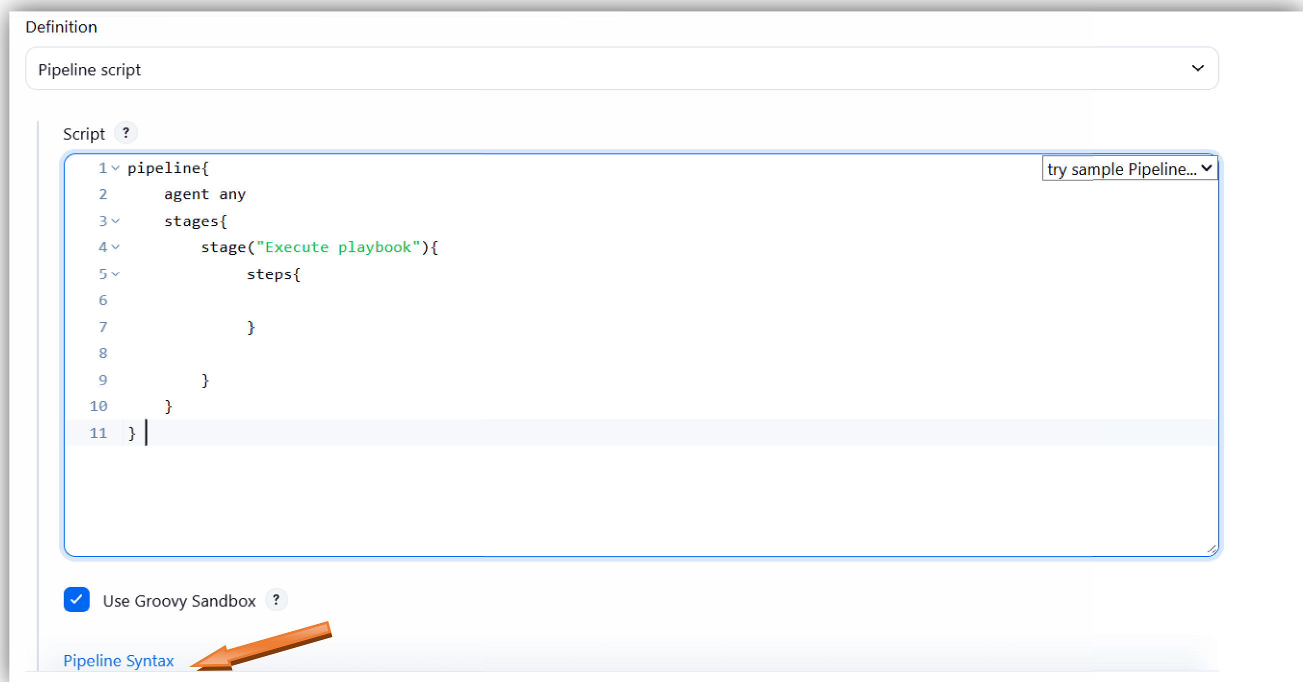


Build after other projects are built :- maven _project1----- > this name is a maven_ssh project name

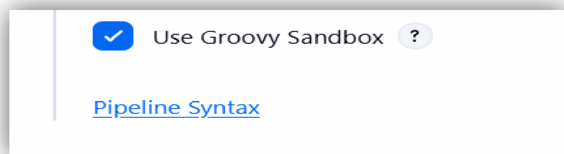


Pipeline definition : - select Pipeline script

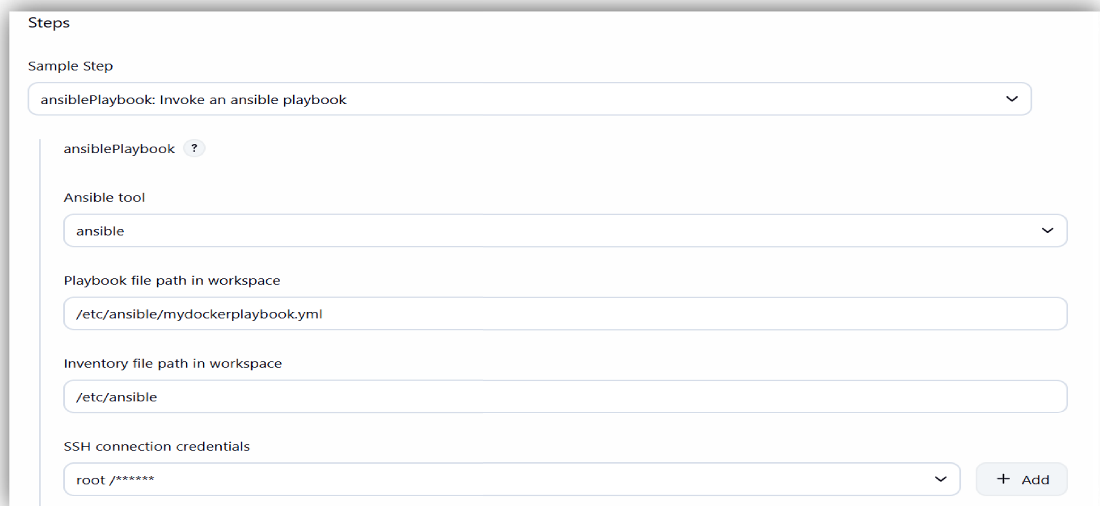
```
        pipeline{
          agent any
          stages{
            stage("Execute playbook"){
               steps{

               }

            }
          }
}
```



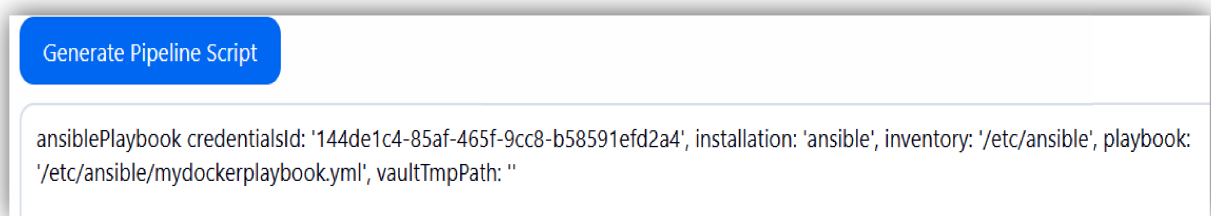
click on :- Pipeline Syntax



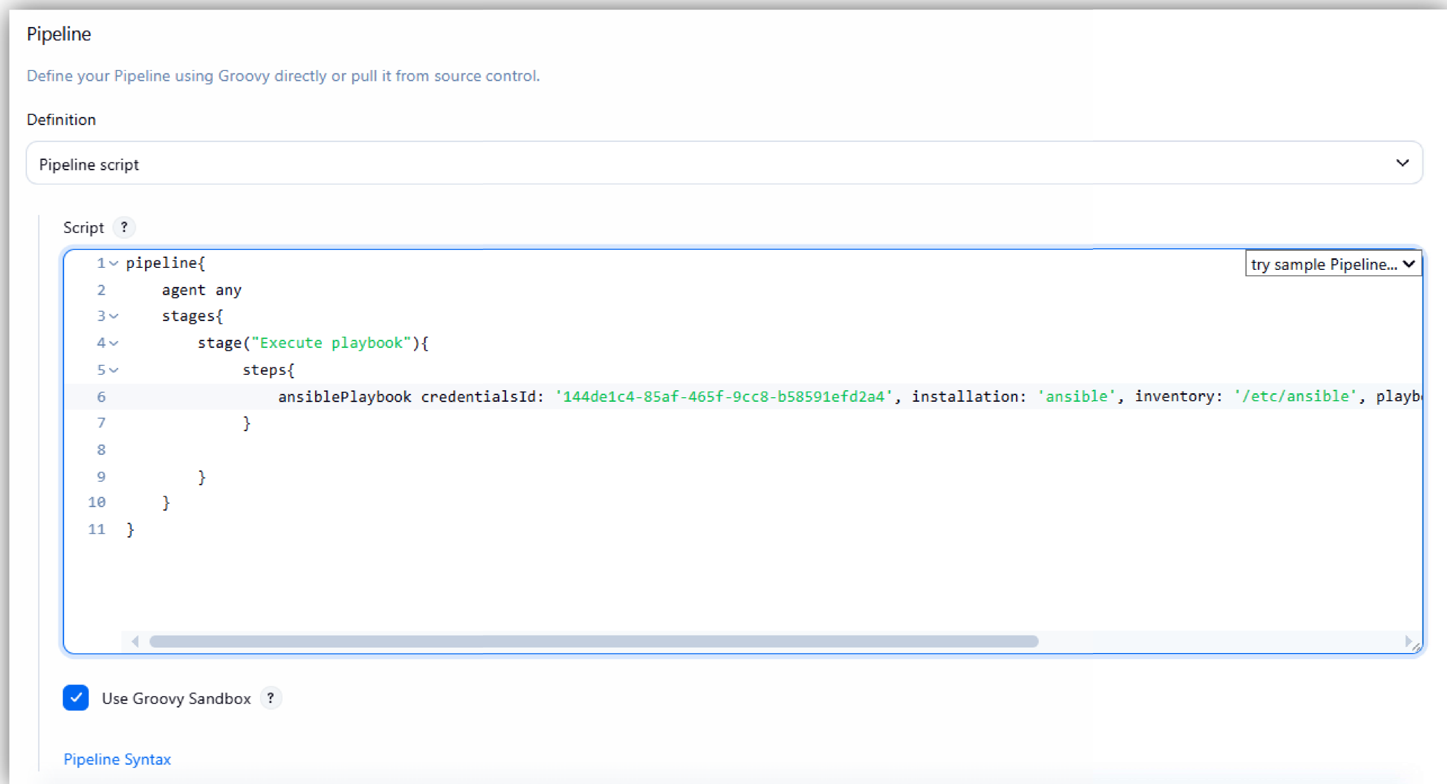
Sample Step select --- > ansibleAdhoc: Invoke an ansible adhoc command



click on generate pipeline script

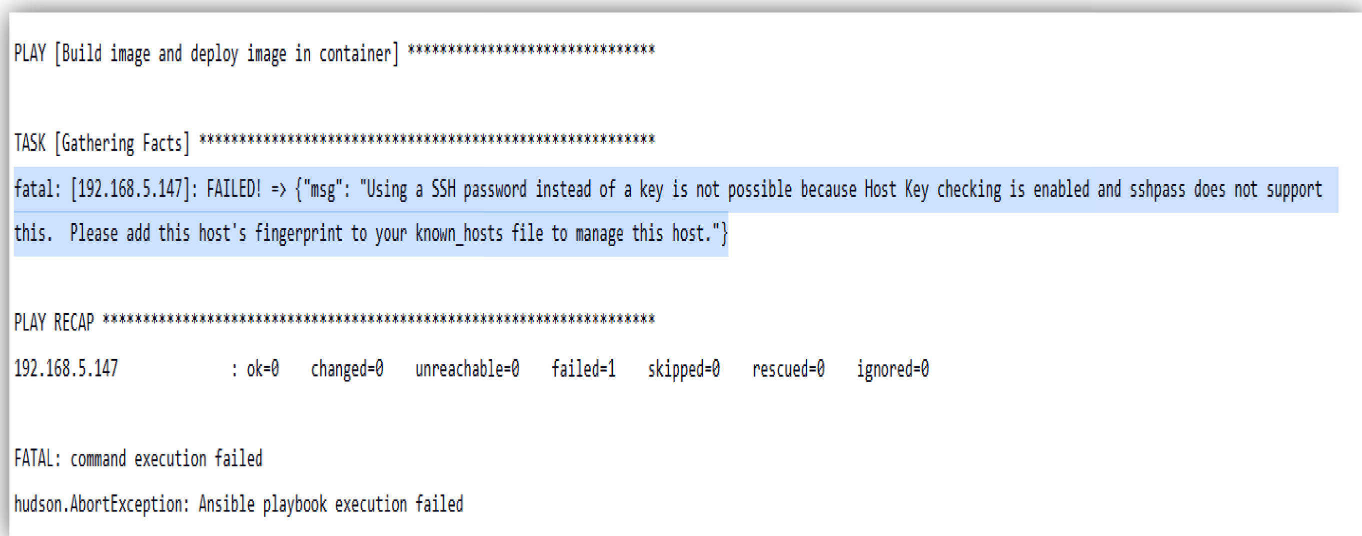


Then copy the script and go back
paste the line number 6



click on Apply and Save
click on build now

if you will get this error :--



** then configure the ansible configuration file

vi /etc/ansible/ansible.cfg

[defaults]
host_key_checking = False

```
# Since Ansible 2.12 (core):
# To generate an example config file (a "disabled" one with all default settings, commented out):
#               $ ansible-config init --disabled > ansible.cfg
#
# Also you can now have a more complete file by including existing plugins:
# ansible-config init --disabled -t all > ansible.cfg

# For previous versions of Ansible you can check for examples in the 'stable' branches of each version
# Note that this file was always incomplete  and lagging changes to configuration settings

# for example, for 2.9: https://github.com/ansible/ansible/blob/stable-2.9/examples/ansible.cfg

[defaults]
host_key_checking = False
```

go to browser jenkins server -------> build Now --- > Successful

go to browser access webapp :- 192.168.5.147:8086/adnan-webapp
Now, testing the CICD
go to Jenkins server and edit the index.jsp
  # cd /root/adnan-webapp
 # vi src/main/webapp/index.jsp

```
<html>
<body>
<h2><%= "Hello World! i have completed CICD pipeline and testing successfull! " %></h2>
</body>
</html>
~
```

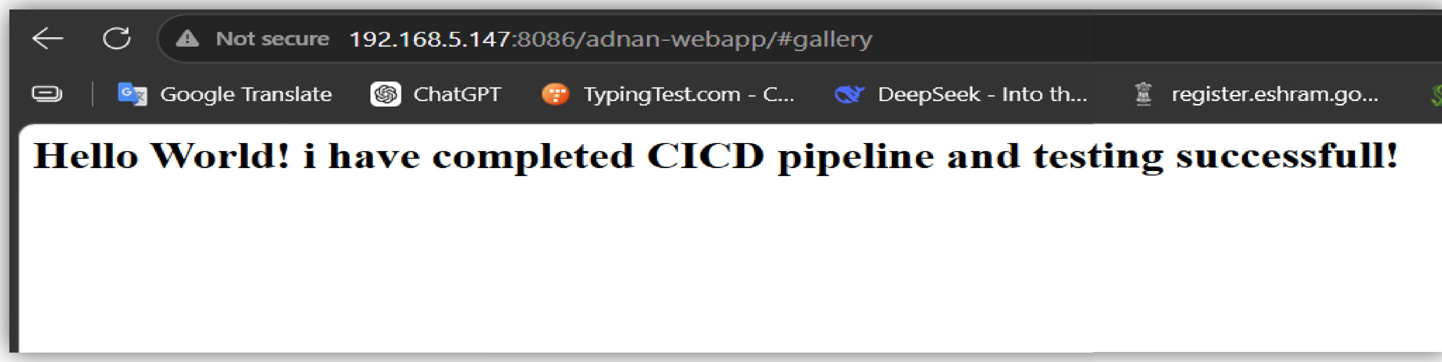
 # git status
git add .

git commit -m "code1 modified for testing”

git push -u origin main

  Username :- adnan
  Password :- <token_number>

go to web page only refresh it


Not secure 192.168.5.147:8086/adnan-webapp/#gallery
Google Translate
ChatGPT
TypingTest.com - C...
DeepSeek - Into th...
register.eshram.go...
Hello World! i have completed CICD pipeline and testing successfull!